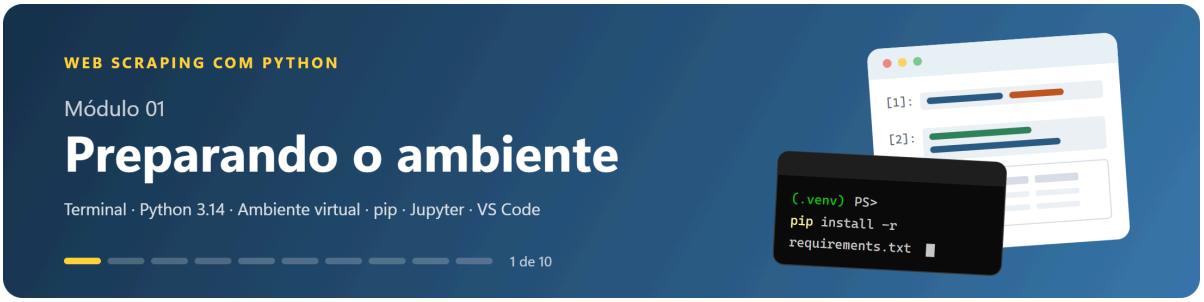


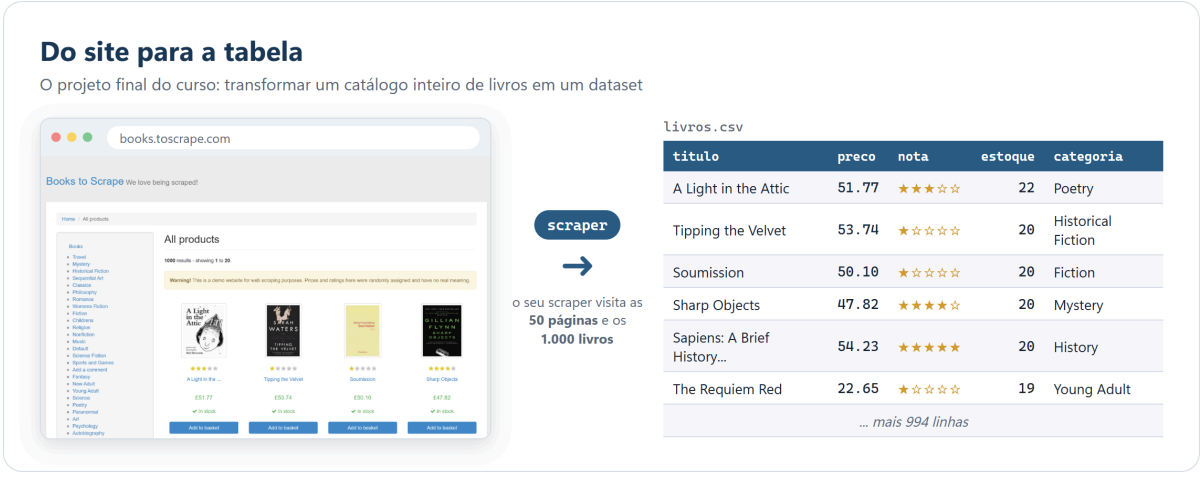
Módulo 01: Preparando o ambiente

Este é o primeiro módulo do curso de **Web Scraping com Python**.

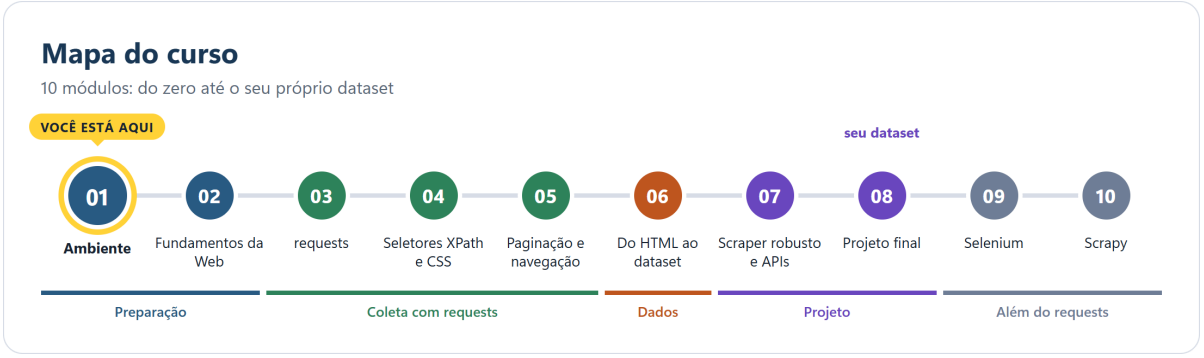
Web scraping é a técnica de escrever um programa que **visita páginas da internet e extrai os dados de interesse**, como preços, nomes, notas e endereços, organizando-os em uma tabela. O que levaria horas de cópia manual, página por página, passa a ser feito de forma automática e padronizada.

Objetivo do curso

Ao final do curso, você terá construído um programa (um *scraper*) que percorre um site inteiro e transforma o que encontrou em um **dataset**, uma tabela de dados pronta para ser analisada:



O curso é dividido em 10 módulos, publicados à medida que ficam prontos. Este é o primeiro:



Neste módulo você vai

- Abrir e usar o **terminal** do seu computador;
- Instalar o **Python 3.14**;
- Criar um **ambiente virtual** para o curso;
- Instalar as **bibliotecas** do curso com o `pip`;
- Abrir este notebook no **Jupyter** (pelo navegador) ou no **VS Code**;
- Entender **células**, **kernel** e **Markdown**, e aprender a ler **mensagens de erro**;
- Rodar um teste que confirma que está **tudo pronto** para o Módulo 02.

Tempo estimado: de 1h a 1h30. A maior parte é instalação, que só se faz uma vez.

Como usar este material

Este módulo tem duas partes:

Parte	O que você faz	Onde
Parte 1: Instalando tudo	Instala o Python e as bibliotecas e abre o Jupyter	Pelo **PDF**, já que o Jupyter ainda não está instalado
Parte 2: Conhecendo o notebook	Executa as primeiras células e aprende a usar a ferramenta	**Dentro do notebook**, no Jupyter ou no VS Code

> **Lendo pelo PDF?** Siga a Parte 1 com o computador ao lado. No final dela, você vai abrir *este mesmo arquivo* no Jupyter e continuar por lá.

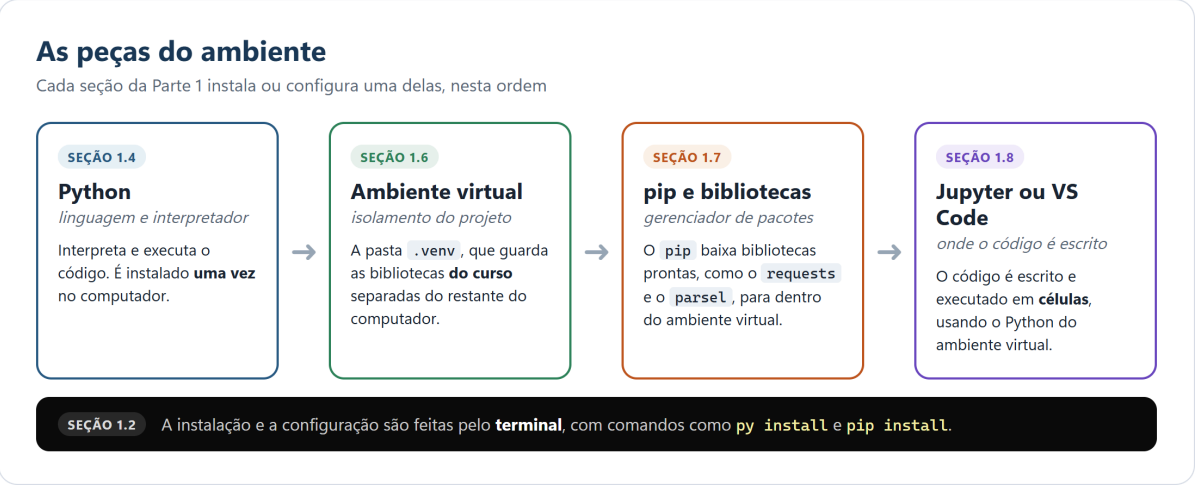
Ao longo do curso, estes símbolos indicam o tipo de cada instrução:

Símbolo	Significado
⌨️	Comando para digitar no **terminal**
▶️	Célula de código para **executar** no notebook
💡	**Dica**
⚠️	**Atenção**: ponto em que costumam acontecer erros
✅	**Checkpoint**: confira se está tudo certo antes de seguir

Parte 1: Instalando tudo

1.1 As peças do ambiente

Antes de instalar, vale entender o papel de cada peça. Cada seção desta Parte 1 trata de uma delas:



- **Python** é a linguagem de programação e também o programa que *executa* o código que escrevemos (o "interpretador").
- O **ambiente virtual** é uma pasta chamada `.venv` que guarda as bibliotecas **usadas neste curso**, isoladas do restante do computador.
- **Bibliotecas** são códigos prontos que adicionam funcionalidades ao Python: o `requests` baixa páginas da internet, o `parsel` localiza informações dentro delas e o `pandas` organiza os dados em tabelas. Elas são instaladas pelo **pip**, o gerenciador de pacotes do Python.
- O **Jupyter** e o **VS Code** são os programas em que o código é escrito e executado, em *notebooks* como este.

A instalação e a configuração de tudo isso são feitas pelo **terminal**.

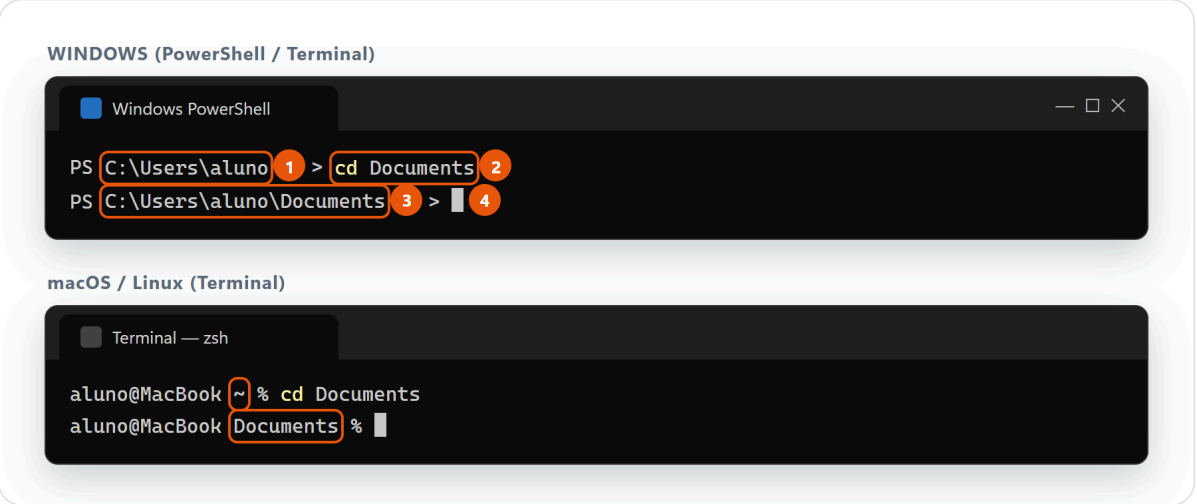
1.2 O terminal

O terminal é um programa em que o computador é operado por **comandos digitados**, em vez de cliques. É a forma padrão de instalar e configurar ferramentas de programação, e neste curso serão necessários poucos comandos.

Como abrir

Sistema	Como abrir o terminal
Windows 10/11	Clique com o botão direito no botão **Iniciar** e escolha **Terminal** (ou **Windows PowerShell**). Também é possível pesquisar "Terminal" no menu Iniciar.
macOS	Aperte `Cmd ⌘ + Espaço`, digite **Terminal** e aperte `Enter`.
Linux	`Ctrl + Alt + T` na maioria das distribuições.

Lendo o terminal



- ① O começo da linha, chamado de **prompt**, mostra **em qual pasta você está**. Todo comando é executado "dentro" dessa pasta.
- ② Você digita um comando e aperta `Enter`. Aqui, `cd Documents` significa "entre na pasta Documents" (*cd* vem de *change directory*).
- ③ O prompt mudou: agora você está **dentro** de `Documents`.
- ④ O cursor piscando indica que o terminal está **esperando** o próximo comando.

Os comandos que você precisa conhecer

O que faz	Windows (PowerShell)	macOS / Linux
Entrar em uma pasta	`cd NomeDaPasta`	`cd NomeDaPasta`
Voltar para a pasta de cima	`cd ..`	`cd ..`
Listar o que tem na pasta atual	`dir`	`ls`

> 💡 **Atalhos:** a tecla `Tab` completa nomes de pastas e arquivos (digite `cd Doc` e aperte `Tab`). A **seta para cima** `↑` traz de volta os comandos anteriores.

> 💡 **Os nomes no terminal nem sempre são os que aparecem na tela.** O Windows traduz alguns nomes apenas na exibição: a pasta mostrada como "Documentos" no Explorador de Arquivos se chama `Documents` no terminal, e a "Área de Trabalho" se chama `Desktop`. O nome da pasta do usuário também pode ser diferente do nome da conta, porque acentos e espaços costumam ser removidos ou trocados: um usuário "João" pode ter a pasta `C:\Users\joao` ou `C:\Users\joao~`, por exemplo. Na dúvida, use `dir` para ver os nomes reais, ou a tecla `Tab` para completá-los. O mesmo acontece no macOS.

1.3 Você já tem o Python?

Muitos computadores já têm Python, ou você pode tê-lo instalado em outra ocasião. Abra o terminal e digite o comando do seu sistema:

⌨️ Windows:

```
python --version
```

⌨️ macOS / Linux:

```
python3 --version
```

Agora compare o que apareceu com a tabela abaixo:

O que o `python --version` respondeu?

Encontre o seu caso e siga a instrução da direita

NO TERMINAL APARECEU...	SIGNIFICA	O QUE FAZER
`Python 3.14.8` (ou 3.11, 3.12, 3.13)	✓ Compatível	Você já tem um Python que serve. **Pule para a seção 1.5.**
`Python 3.15.0` (ou mais novo)	! Novo demais	Algumas bibliotecas ainda não suportam essa versão. **Instale também o 3.14 (seção 1.4)**: as duas versões convivem sem problema.
`Python 3.9.6` (ou 3.10 e anteriores)	! Antigo demais	As bibliotecas do curso precisam do 3.11 ou mais novo. **Instale o 3.14 (seção 1.4).**
`Python was not found; run without arguments to install from the Microsoft Store...` ou `O termo 'python' não é reconhecido...` ou `command not found: python3`	✕ Não instalado	O computador não tem Python (ou não o encontra). **Instale o 3.14 (seção 1.4).**

> 💡 No macOS, pode aparecer uma janela oferecendo instalar as "ferramentas de linha de comando". Ela pode ser cancelada: essas ferramentas trazem um Python antigo. Siga a seção 1.4.

> **Por que não usar a versão mais nova, a 3.15?** O Python 3.15 foi lançado em outubro de 2026, e algumas bibliotecas usadas no curso ainda não publicaram versões compatíveis com ele. Até isso acontecer, a instalação pode falhar com erros de compilação. É uma situação comum: **as bibliotecas costumam levar alguns meses para acompanhar uma nova versão do Python**. Por isso o curso usa o **3.14**, que é estável e totalmente suportado.

1.4 Instalando o Python 3.14

Siga apenas o roteiro do seu sistema operacional.

Windows

Desde o Python 3.14, a forma recomendada de instalar o Python no Windows é o **Python install manager**, um programa que baixa e gerencia as versões do Python instaladas no computador.

Passo 1. Acesse **python.org/downloads** e clique em **Download Python install manager**:



Passo 2. Abra o arquivo baixado (`python-manager-....msix`) e clique em **Instalar**.

Passo 3. Na primeira vez que roda, o install manager abre um **assistente de configuração** no terminal, em inglês. Ele pode fazer algumas perguntas. As respostas recomendadas são:

Se aparecer a pergunta...	Responda	Por quê
`Update setting now? [y/N]` (sobre *paths longer than 260 characters*)	**y** se o computador for seu; **n** se você não tiver senha de administrador	Libera nomes de pasta longos. Ajuda, mas não é obrigatório.
`Add commands directory to your PATH now? [y/N]`	**y**	Permite usar comandos como `python3.14` no terminal.
`Install CPython now? [Y/n]` (sobre *the current latest version*)	**n**	Instalaria a 3.15. A 3.14 será instalada no próximo passo.

> 💡 Essas respostas podem ser alteradas depois: o assistente pode ser executado novamente com o comando `py install --configure` .

Passo 4. Feche o terminal, abra um **novo** e instale o Python 3.14:

⌨

```
py install 3.14
```

Passo 5. Confira se funcionou:

⌨

```
py -3.14 --version
```

Deve aparecer `Python 3.14` seguido de mais um número, por exemplo `Python 3.14.8` .

Plano B: não consegui usar o install manager (computador da escola ou do trabalho, sem Microsoft Store etc.)
Use o instalador tradicional:

1. Acesse python.org/downloads/windows e procure a versão **Python 3.14** mais recente da lista.
2. Clique em **Windows installer (64-bit)**. Se o seu computador tiver processador ARM (por exemplo, Snapdragon), escolha **Windows installer (ARM64)**.
3. Na primeira tela do instalador, **marque a caixa "Add python.exe to PATH"** (deixá-la desmarcada é a causa mais comum de problemas) e clique em **Install Now**.
4. Abra um terminal novo e confira com `python --version`.

O instalador tradicional ainda funciona para o 3.14, mas está sendo descontinuado e não existirá mais a partir do Python 3.16.

macOS

Passo 1. Acesse **python.org/downloads/macos** e procure a versão **Python 3.14** mais recente da lista. Clique no link **macOS installer** logo abaixo dela.

Passo 2. Abra o arquivo `.pkg` baixado e avance com **Continuar**, **Concordo** e **Instalar**. O Mac vai pedir a sua senha.

Passo 3. Abra um terminal **novo** e confira:

⌨️

```
python3.14 --version
```

Deve aparecer `Python 3.14` seguido de mais um número.

> ⚠️ No macOS, use sempre `python3.14` ou `python3`. O comando `python` sozinho normalmente não existe.

Linux (Ubuntu, Debian, Mint)

O Linux quase sempre já vem com Python. Instale os pacotes que faltam para criar ambientes virtuais:

⌨️

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

Depois confira com `python3 --version`. Qualquer versão entre **3.11 e 3.14** serve para o curso.

> 💡 Usa outra distribuição (Fedora, Arch...)? Os pacotes têm nomes parecidos: procure por `python3` e `python3-pip` no gerenciador de pacotes dela.

1.5 A pasta do curso

O curso fica no GitHub, em **github.com/Johaan01/curso-web-scraping**. Baixe-o como preferir, pelo botão **Code** da página (**Download ZIP**) ou com o `git clone`, e coloque a pasta em um lugar fácil de achar, como **Documentos**. Pelo ZIP, a pasta vem com o nome `curso-web-scraping-main`: renomeie para `curso-web-scraping`, para seguir os comandos exatamente como estão neste módulo.

```
Documentos/
└── curso-web-scraping/
    ├── 01-ambiente/              ← você está aqui
    │   ├── 01-ambiente.ipynb
    │   ├── 01-ambiente.pdf
    │   └── imagens/
    ├── 02-fundamentos-da-web/
    ├── 03-requests/
    ├── 04-seletores-xpath-css/
    ├── 05-paginacao-e-navegacao/
    ├── README.md
    └── requirements.txt          ← a lista de bibliotecas do curso
```

> ⚠️ **Usa o OneDrive?** Se a sua pasta Documentos é sincronizada com o OneDrive (aparece um ícone de nuvem ao lado dos arquivos), coloque o curso direto na sua pasta de usuário, por exemplo `C:\Users\seu-nome\curso-web-scraping`. Assim o OneDrive não tenta sincronizar os milhares de arquivos que o ambiente virtual vai criar.

Agora abra o terminal **dentro dessa pasta**. Há duas formas:

- **Windows 11:** abra a pasta `curso-web-scraping` no Explorador de Arquivos, clique com o **botão direito** em um espaço vazio e escolha **Abrir no Terminal**.
- **Qualquer sistema:** abra o terminal e use o `cd` até chegar lá:

⌨️ Windows:

```
cd Documents\curso-web-scraping
```

⌨️ macOS / Linux:

```
cd Documents/curso-web-scraping
```

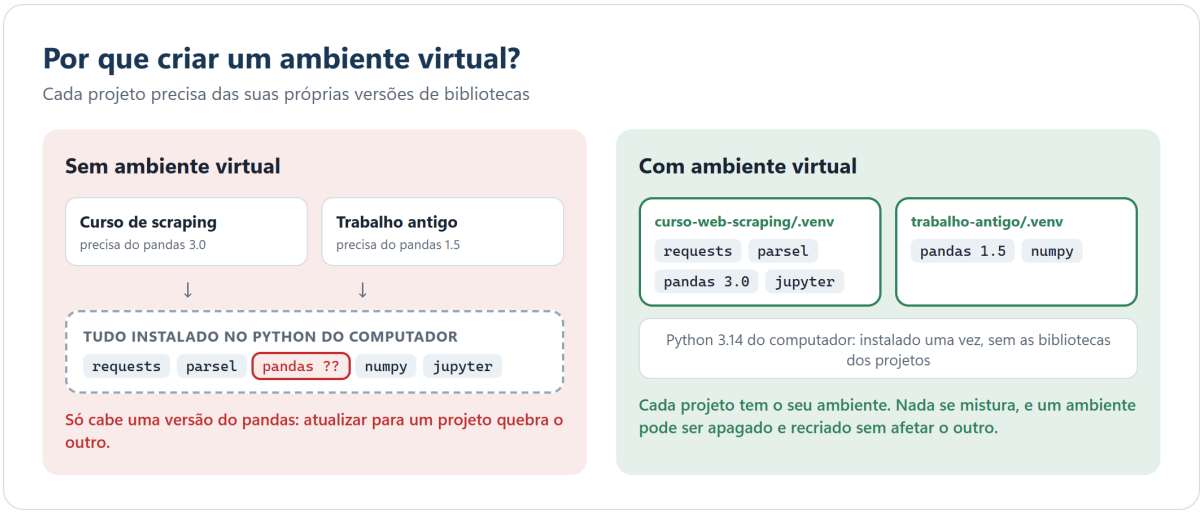
> 💡 Outra opção: digite `cd ` (com um espaço no final) e **arraste a pasta** do Explorador de Arquivos ou do Finder para dentro da janela do terminal. O caminho completo é preenchido automaticamente; em seguida, aperte `Enter`.

✅ **Checkpoint:** digite `dir` (Windows) ou `ls` (macOS/Linux). Na lista deve aparecer o arquivo `requirements.txt`. Se ele aparecer, o terminal está na pasta certa.

1.6 Criando o ambiente virtual

Por que isolar as bibliotecas?

Projetos diferentes costumam depender de versões diferentes das mesmas bibliotecas. Se tudo for instalado diretamente no Python do computador, atualizar uma biblioteca para um projeto pode quebrar outro. O **ambiente virtual** resolve isso dando a cada projeto o seu próprio conjunto de bibliotecas:



Na prática, o ambiente virtual é uma pasta chamada `.venv` dentro da pasta do curso, onde ficam todas as bibliotecas instaladas. Se algo der errado, basta apagar a pasta e criá-la de novo.

Criando

Com o terminal **na pasta do curso** (seção 1.5), digite o comando do seu sistema:

Sistema	⌨️ Comando
Windows, se instalou pela seção 1.4	`py -3.14 -m venv .venv`
Windows, se já tinha Python 3.11 a 3.14	`python -m venv .venv`
macOS	`python3.14 -m venv .venv`
Linux	`python3 -m venv .venv`

O que cada parte do comando significa:



O comando leva alguns segundos e **não exibe nenhuma mensagem** quando é bem-sucedido, o que é comum entre comandos de terminal. Para conferir, liste a pasta: deve aparecer uma pasta nova chamada `.venv`. No Windows, use `dir`; no macOS e no Linux, use `ls -a` (pastas cujo nome começa com ponto ficam escondidas no `ls` comum).

Ativando

Depois de criado, o ambiente precisa ser **ativado** para que o terminal passe a usar o Python e as bibliotecas que estão dentro dele.

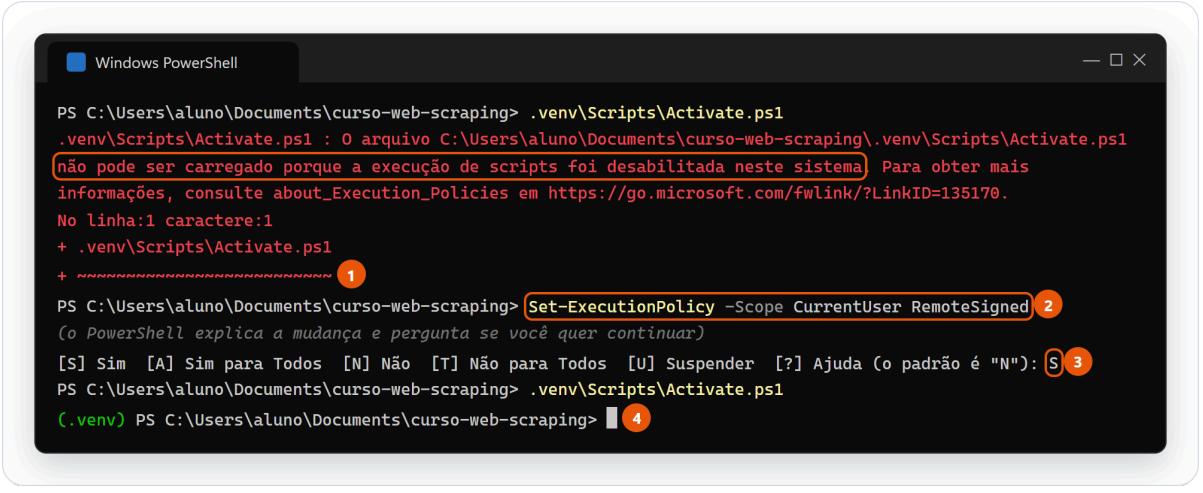
Sistema / terminal	⌨️ Comando para ativar
Windows: PowerShell ou Terminal	`.venv\Scripts\Activate.ps1`
Windows: Prompt de Comando (cmd)	`.venv\Scripts\activate.bat`
macOS / Linux	`source .venv/bin/activate`

Quando a ativação funciona, aparece `(.venv)` no começo da linha do terminal:



- ① O comando que cria o ambiente. A ausência de mensagem indica que deu certo.
- ② O comando que ativa o ambiente.
- ③ A confirmação da ativação: `(.venv)` no começo da linha.

> ⚠️ **Erro comum no Windows:** se aparecer uma mensagem em vermelho dizendo que a *"execução de scripts foi desabilitada neste sistema"*, o Windows está bloqueando scripts por segurança. Veja na imagem abaixo como liberar **apenas para o seu usuário**:



- ① O erro: o Windows bloqueou o script de ativação.
- ② O comando que libera scripts para o seu usuário: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- ③ O PowerShell pede confirmação: digite **S** (ou **Y**, se o Windows estiver em inglês) e aperte `Enter`.
- ④ Na nova tentativa, a ativação funciona e o `(.venv)` aparece.

> 💡 **Importante:** a ativação vale só para **aquela janela** do terminal. Sempre que abrir um terminal novo para trabalhar no curso, entre na pasta e ative o ambiente de novo. Para desativar, digite `deactivate`.

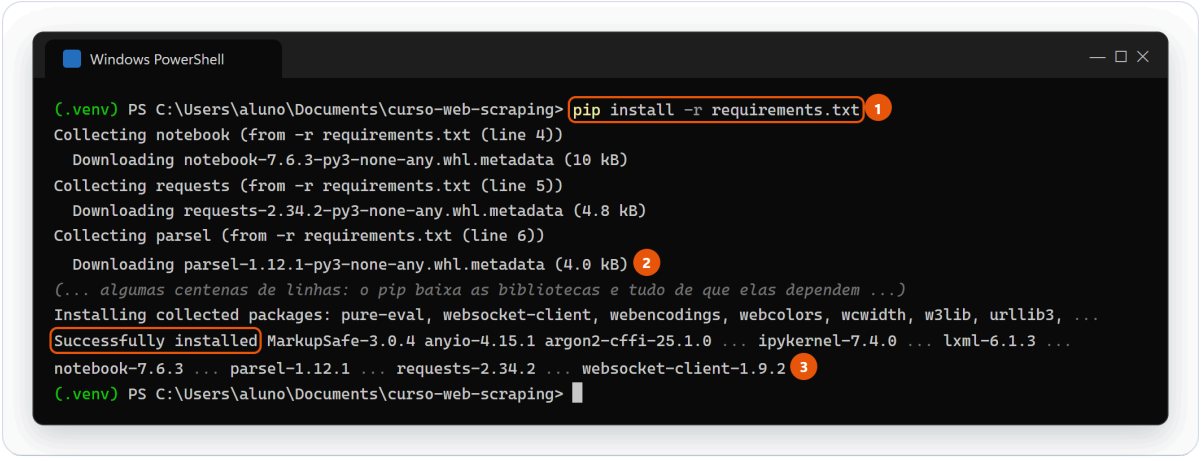
1.7 Instalando as bibliotecas com o pip

Com o ambiente **ativado** (o `(.venv)` aparecendo no terminal), instale as bibliotecas do curso. O arquivo `requirements.txt` contém a **lista de dependências** do curso: o `pip` lê essa lista e instala tudo de uma vez.

⌨️

```
pip install -r requirements.txt
```

O `-r` significa "leia a lista deste arquivo". O pip vai baixar as bibliotecas e mais algumas dezenas de outras de que elas dependem. Isso leva de 1 a 5 minutos, dependendo da sua internet.



- ① O comando de instalação (repare no `(.venv)` antes dele).
- ② O pip mostra cada biblioteca que encontra e baixa.
- ③ Quando termina, a última mensagem começa com `Successfully installed`, seguida do nome de todos os pacotes instalados.

Estas são as bibliotecas usadas nos Módulos 01 a 05, e onde cada uma entra:

Biblioteca	Para que serve	Módulos
`notebook`	O Jupyter Notebook, onde escrevemos e executamos o código	01 em diante
`requests`	Faz pedidos HTTP: baixa páginas e dados da internet	02 em diante
`parsel`	Encontra os dados dentro do HTML, com XPath e seletores CSS	03 em diante
`lxml`	Instalada junto com o `parsel`: é ela que lê o HTML por trás dele	03 em diante

O restante do que os módulos usam (`json`, `pathlib`, `time`, `urllib`) já vem com o Python. Os Módulos 06 em diante vão precisar de mais algumas bibliotecas, como o `pandas`. Quando eles forem publicados, essas bibliotecas entram no `requirements.txt`: basta baixar a versão nova do curso e rodar o `pip install -r requirements.txt` de novo.

> ⚠️ Se aparecer `ERROR` em vermelho no final, leia a última linha da mensagem e consulte a seção **Problemas comuns**, no final deste módulo. Já as mensagens que começam com `[notice]` (por exemplo, *A new release of pip is available*) são apenas avisos e podem ser ignoradas.

1.8 Abrindo este notebook

Com as bibliotecas instaladas, o próximo passo é abrir este mesmo arquivo, o `01-ambiente.ipynb`, em um programa que executa notebooks. Escolha **um** dos caminhos; os dois funcionam igualmente bem no curso:

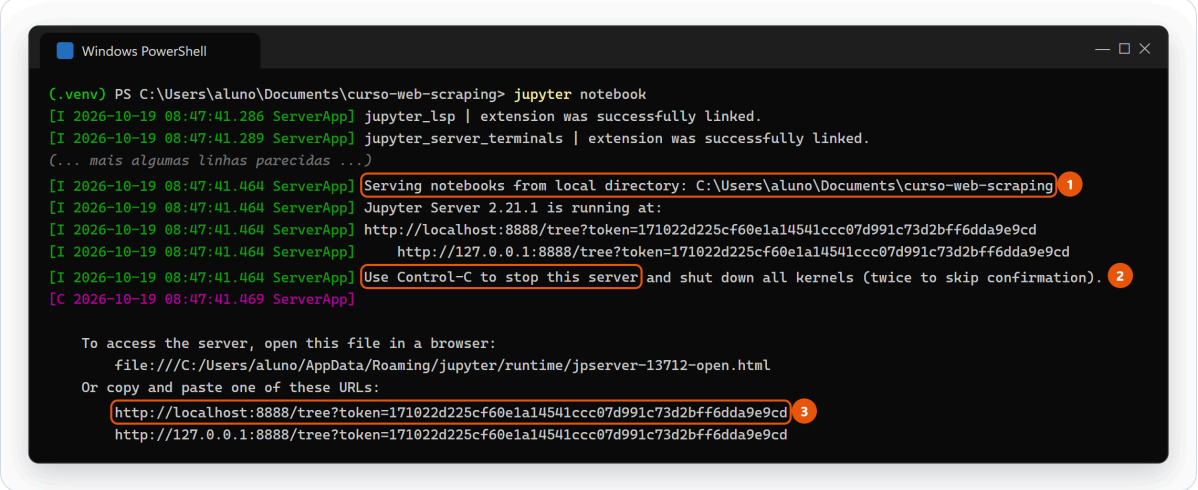
	Caminho A: Jupyter no navegador	Caminho B: VS Code
Como é	Abre como um site, no seu navegador	Um editor de código completo, instalado no computador
Bom para	Quem quer começar rápido	Quem já usa ou quer aprender a usar o VS Code
Precisa instalar	Nada: ele veio no `requirements.txt`	O VS Code e duas extensões

Caminho A: Jupyter no navegador

Passo 1. No terminal, na pasta do curso e com o `(.venv)` ativo, digite:

```
jupyter notebook
```

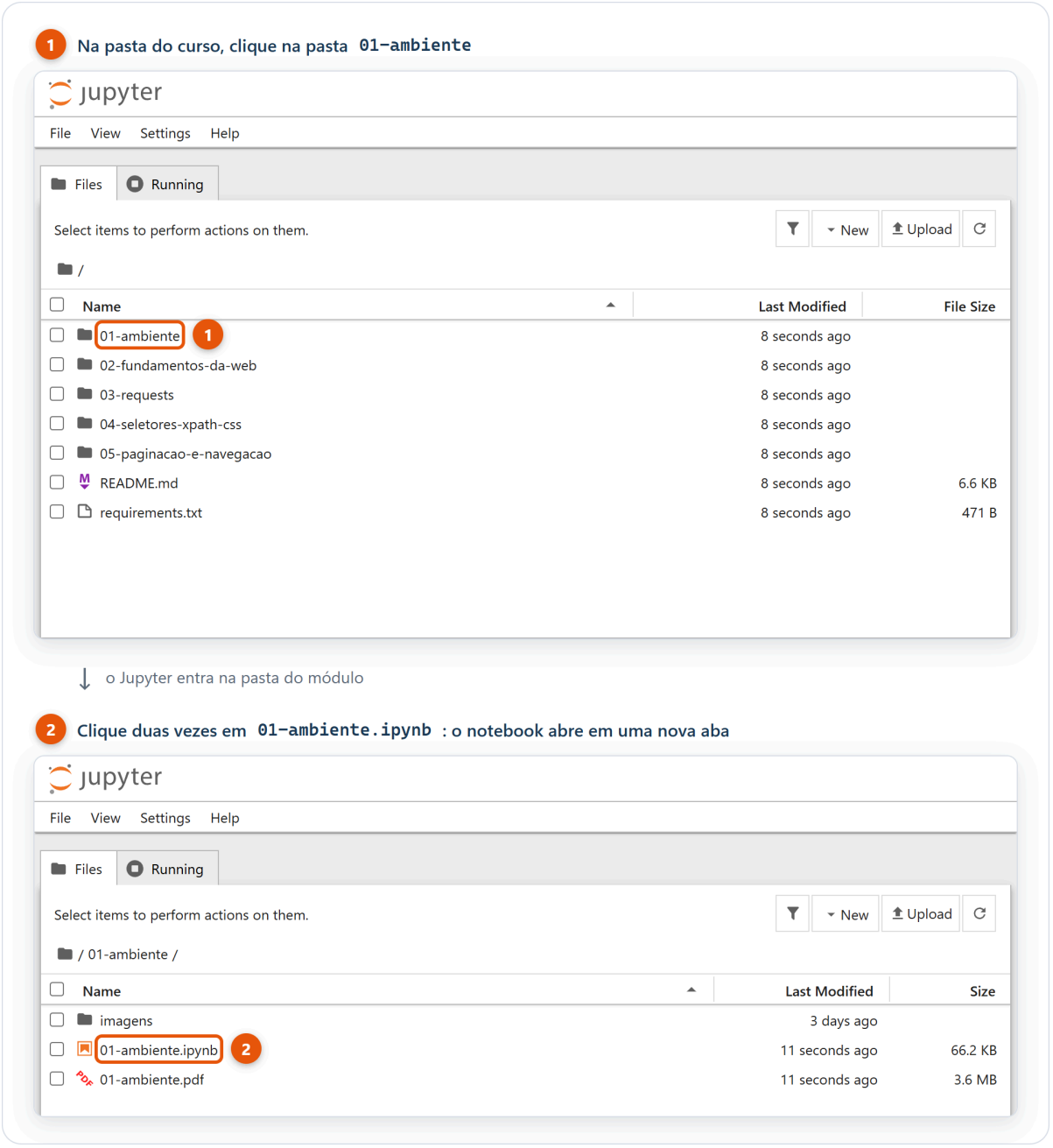
O terminal mostra várias mensagens e, em alguns segundos, o navegador abre sozinho.



- ① A pasta que o Jupyter vai mostrar no navegador. Deve ser a pasta do curso.
- ② Como encerrar o Jupyter depois: `Ctrl + C` neste terminal.
- ③ O endereço do Jupyter. Se o navegador não abrir sozinho, copie esse endereço e cole no navegador.

> ⚠️ **Não feche esse terminal.** Ele precisa ficar aberto (pode ser minimizado) enquanto o Jupyter estiver em uso: é ele que executa o código, e o navegador é apenas a interface.

Passo 2. O navegador mostra os arquivos da pasta do curso. Clique na pasta `01-ambiente` ① e depois dê um **clique duplo** em `01-ambiente.ipynb` ②. O notebook abre em uma **nova aba** do navegador:



Passo 3. Com o notebook aberto, identifique as partes principais da tela, usadas na Parte 2:



- ① **Menus**: tudo o que o Jupyter faz está aqui (salvar, executar, reiniciar...).
- ② **Barra de ferramentas**: atalhos para as ações mais usadas, como o ▶️ que executa a célula selecionada.
- ③ **Tipo da célula**: mostra se a célula selecionada é de código (*Code*) ou de texto (*Markdown*).
- ④ **Kernel**: o processo que executa o código. O círculo vazio ◯ significa *livre*; cheio ●, *ocupado executando*.
- ⑤ **Célula selecionada**: marcada pela barra azul à esquerda.

Para encerrar o Jupyter no fim da aula: salve o notebook (`Ctrl + S`) e use o menu **File → Shut Down**. Outra opção é apertar `Ctrl + C` **duas vezes** no terminal.

Caminho B: VS Code

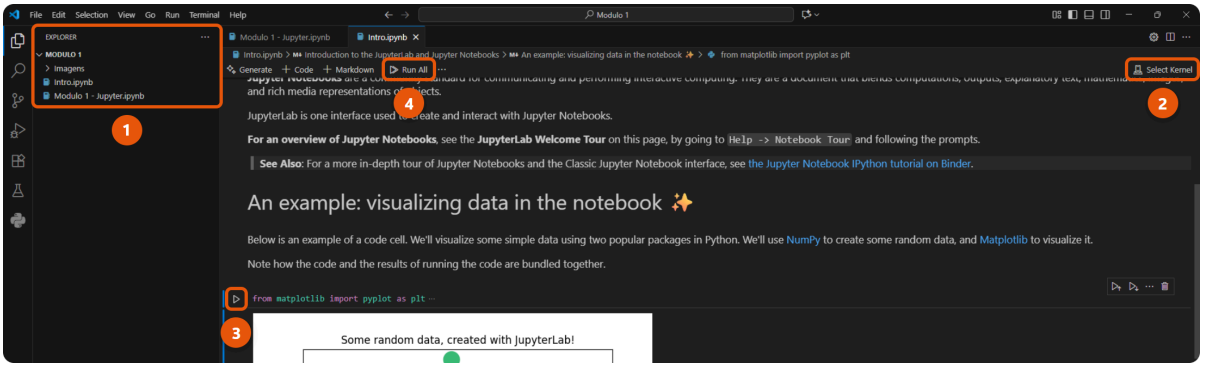
Passo 1. Baixe e instale o VS Code em **code.visualstudio.com**. No Windows, durante a instalação, deixe marcada a opção **Add to PATH**.

Passo 2. Abra o VS Code e instale as extensões **Python** e **Jupyter**, ambas publicadas pela **Microsoft**: clique no ícone de Extensões na barra lateral (ou aperte `Ctrl + Shift + X`), pesquise o nome de cada uma e clique em **Install**.

Passo 3. Abra a pasta do curso pelo menu **File → Open Folder...** e escolha `curso-web-scraping`. Se o VS Code perguntar se você confia nos autores dos arquivos, responda **Yes, I trust the authors**.

Passo 4. Na barra lateral (*Explorer*), abra a pasta `01-ambiente` e clique em `01-ambiente.ipynb`.

Passo 5. Escolha o **kernel**, ou seja, qual Python vai executar o notebook: clique em **Select Kernel**, no canto superior direito, depois em **Python Environments...** e escolha o `.venv` (o VS Code costuma marcá-lo como *Recommended*).



- ① **Explorer**: os arquivos da pasta aberta.
- ② **Select Kernel**: onde você escolhe o `.venv`. Depois de escolhido, o botão passa a mostrar o nome do ambiente.
- ③ **Executar célula**: o ▷ ao lado de cada célula de código.

- ④ **Run All**: executa todas as células. No menu ⋯ ao lado ficam outras ações, como **Restart** (reiniciar o kernel).

> ⚠️ Se o `.venv` não aparecer na lista, confira se você abriu a pasta `curso-web-scraping` (e não a `01-ambiente`) no Passo 3. Depois feche e abra o VS Code de novo.

> 💡 O VS Code tem um terminal embutido (menu **Terminal** → **New Terminal**), que já abre na pasta do curso e costuma ativar o `.venv` sozinho.

✅ **Checkpoint da Parte 1:** este notebook está aberto no Jupyter ou no VS Code. **A partir daqui, continue dentro do notebook**: a Parte 2 é prática, e as células devem ser executadas à medida que você lê.

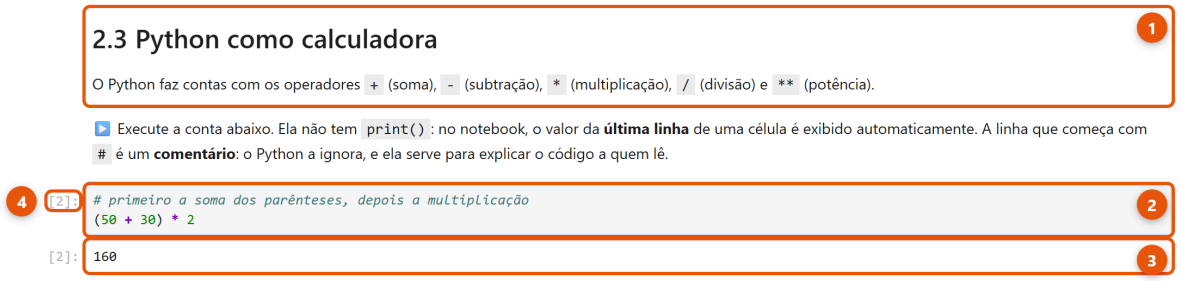
Parte 2: Conhecendo o notebook

A partir daqui, o módulo funciona como todos os seguintes: uma explicação curta e, logo abaixo, uma célula curta para executar. Leia cada explicação, execute a célula e só então siga para a próxima.

Os exercícios, marcados como **Sua vez**, são divididos em passos. Cada passo tem uma célula vazia, onde você escreve o código, e a resposta oculta logo abaixo: tente resolver antes de consultá-la.

2.1 Células: os blocos do notebook

Um notebook é uma sequência de **células**. Existem dois tipos principais:



- ① **Célula de texto (Markdown)**, como esta que você está lendo: serve para explicar, organizar e documentar.
- ② **Célula de código**: guarda código Python.
- ③ **Saída**: quando a célula de código é executada, o resultado aparece logo abaixo dela.
- ④ **Contador de execução**: o número entre colchetes, como `[1]`, mostra **em que ordem** as células foram executadas. Uma célula que ainda não foi executada mostra `[ ]`.

2.2 Executando sua primeira célula

Para executar uma célula de código, clique nela e use **uma** destas formas:

Forma	O que acontece
`Shift + Enter`	Executa e **vai para a próxima** célula (a mais usada)
`Ctrl + Enter`	Executa e **continua** na mesma célula
Botão ▶️ (Jupyter) ou ▷ (VS Code)	Executa com o mouse

A célula abaixo usa o `print()`, que **mostra na tela** o que estiver entre os parênteses. Textos ficam entre aspas.

▶️ Clique na célula e aperte `Shift + Enter`:

```
In [1]: print("Olá! Meu primeiro código no Jupyter está funcionando.")
```

```
Olá! Meu primeiro código no Jupyter está funcionando.
```

A frase aparece logo abaixo da célula, e o `[ ]` à esquerda passa a mostrar um número.

> 💡 Enquanto uma célula está executando, aparece `[*]` no lugar do número. Quando o número aparece, a execução terminou.

Sua vez: o seu primeiro print

Use o `print()` para mostrar uma frase com o seu nome, por exemplo: `Meu nome é Ana e estou aprendendo web scraping.`

```
In [2]: # Escreva o seu código aqui
```

Ver a resposta

```
print("Meu nome é Ana e estou aprendendo web scraping.")
```

2.3 Python como calculadora

O Python faz contas com os operadores `+` (soma), `-` (subtração), `*` (multiplicação), `/` (divisão) e `**` (potência).

▶️ Execute a conta abaixo. Ela não tem `print()`: no notebook, o valor da **última linha** de uma célula é exibido automaticamente. A linha que começa com `#` é um **comentário**: o Python a ignora, e ela serve para explicar o código a quem lê.

In [3]:
```
# primeiro a soma dos parênteses, depois a multiplicação
(50 + 30) * 2
```

Out[3]:
```
160
```

▶️ A divisão com `/` sempre dá um número com casas decimais:

In [4]:
```
10 / 4
```

Out[4]:
```
2.5
```

▶️ A potência usa `**`. Calcule 2 elevado a 10:

In [5]:
```
2 ** 10
```

Out[5]:
```
1024
```

Quando uma célula tem várias contas, só o valor da última é exibido. Para mostrar mais de um resultado na mesma célula, use um `print()` para cada um.

▶️ Execute:

In [6]:
```
print(10 / 4)
print(7 - 12)
```

```
2.5
-5
```

Sua vez: contas

Passo 1 de 2. Um site tem 35 páginas, com 24 produtos em cada uma. Quantos produtos ele tem?

In [7]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
35 * 24
```

Passo 2 de 2. Um scraper espera 0,5 segundo entre um pedido e outro. Quantos segundos ele leva para pedir as 35 páginas? No Python, a vírgula decimal é escrita com ponto: `0.5`.

In [8]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
35 * 0.5
```

2.4 Variáveis e a memória do kernel

Uma **variável** é um nome associado a um valor. Variáveis são criadas com o sinal `=`.

▶️ Crie três variáveis. A célula apenas guarda valores, por isso **não exibe nada**:

```
In [9]: site = "books.toscrape.com"
        paginas = 50
        livros_por_pagina = 20
```

▶️ Escrever só o nome de uma variável mostra o valor dela:

```
In [10]: site
Out[10]: 'books.toscrape.com'
```

As variáveis criadas em uma célula **continuam existindo nas seguintes**. Quem as guarda é o **kernel**, o processo que executa o código e mantém a memória do notebook.

▶️ Calcule o total de livros usando as variáveis da célula anterior:

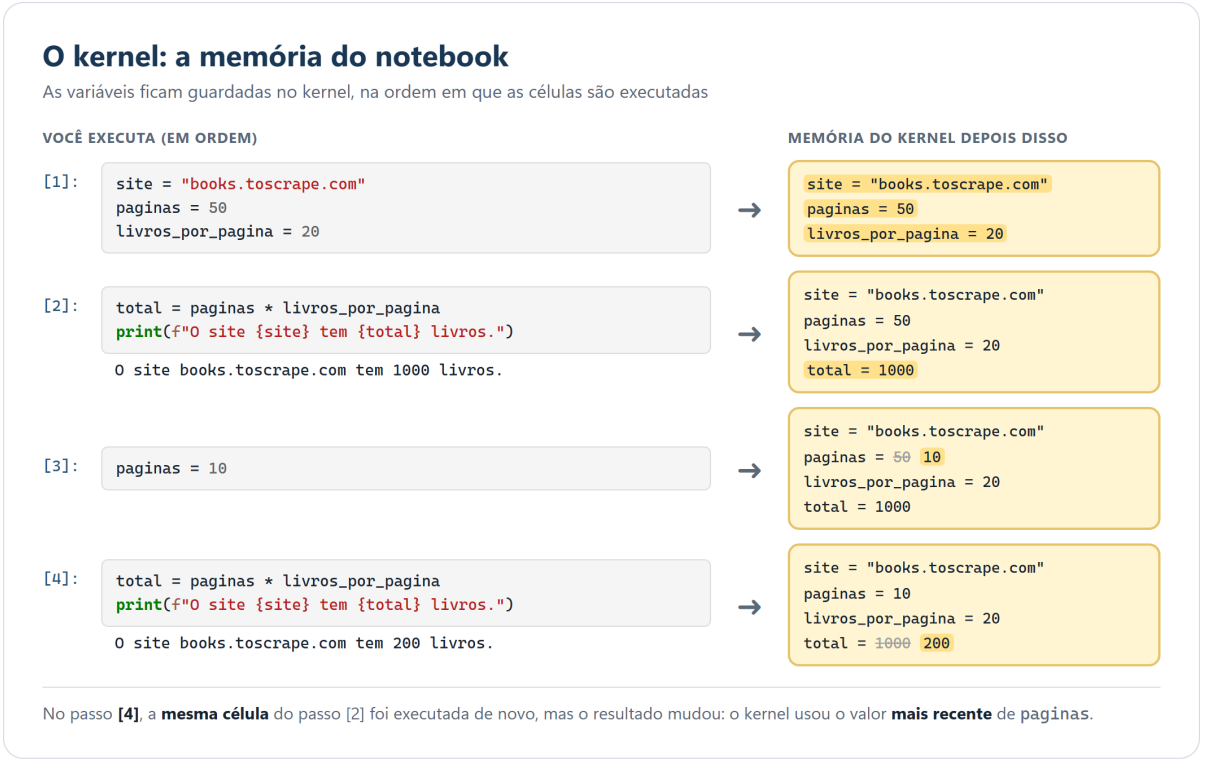
```
In [11]: total = paginas * livros_por_pagina
         total
Out[11]: 1000
```

▶️ Mostre uma frase com os valores. O `f` antes das aspas cria uma **f-string**: tudo o que estiver entre `{chaves}` é trocado pelo valor da variável. Esse recurso será usado ao longo de todo o curso.

```
In [12]: print(f"O site {site} tem {total} livros.")
O site books.toscrape.com tem 1000 livros.
```

A ordem de execução importa

O kernel não lê o notebook de cima para baixo. Ele executa as células **na ordem em que você as executa**. Se uma variável é alterada em uma célula mais abaixo e, depois, uma célula de cima é executada de novo, ela usa o valor novo. O diagrama mostra o que acontece na memória:



▶️ Faça o teste. Mude o valor de `paginas`:

```
In [13]: paginas = 10
```

▶️ Agora calcule o total de novo:

In [14]:
```
total = paginas * livros_por_pagina
total
```

Out[14]: 200

O total passou a ser `200`, porque o kernel agora guarda `paginas = 10`. Se você voltar e executar de novo a célula da frase, mais acima, ela também vai dizer 200 livros, e o contador dela vai ficar **maior** que o destas células: ela foi executada **depois**.

> 💡 **Boa prática:** se o notebook passar a mostrar resultados inesperados, reinicie o kernel e execute tudo de novo, de cima para baixo. No Jupyter: menu **Kernel → Restart Kernel and Run All Cells...**. No VS Code: **Restart** e depois **Run All**.

Sua vez: variáveis

Passo 1 de 3. Crie a variável `paginas_loja`, com o valor `35`, e a variável `produtos_por_pagina`, com o valor `24`.

In [15]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
paginas_loja = 35
produtos_por_pagina = 24
```

Passo 2 de 3. Calcule o total de produtos e guarde o resultado na variável `total_loja`.

In [16]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
total_loja = paginas_loja * produtos_por_pagina
```

Passo 3 de 3. Mostre, com uma f-string, a frase `A loja tem 840 produtos.`

In [17]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
print(f"A loja tem {total_loja} produtos.")
```

Pense antes de executar

Sem executar nada, responda: se as células abaixo forem executadas na ordem **C, depois A, depois B**, o que será mostrado?

- Célula A: `x = 5`
- Célula B: `print(x * 2)`
- Célula C: `x = 100`

Ver a resposta

`10`. A célula C guardou `x = 100`, mas logo depois a célula A trocou o valor para `5`. Quando a B foi executada, `x` valia `5`, e `5 * 2 = 10`. O kernel sempre usa o valor **mais recente**.

2.5 Reiniciando o kernel

Reiniciar o kernel apaga toda a memória: as variáveis deixam de existir. Isso é útil quando a execução trava ou quando o estado do notebook fica inconsistente.

▶️ Faça o teste:

1. Reinicie o kernel. No Jupyter: menu **Kernel** → **Restart Kernel...** (ou o botão ↻ da barra de ferramentas). No VS Code: botão **Restart** na barra de cima.
2. Execute a célula abaixo **sem** executar as anteriores.

```
In [1]: print(total)
```

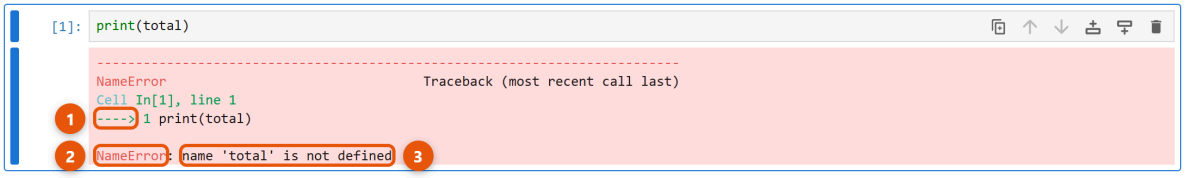
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 1
----> 1 print(total)

NameError: name 'total' is not defined
```

O resultado é um **erro**, que termina com `NameError: name 'total' is not defined` ("o nome `total` não está definido"). Após a reinicialização, o kernel perdeu todas as variáveis, inclusive `total`. Para que ela volte a existir, é preciso executar de novo as células que a criam.

2.6 Lendo mensagens de erro

Erros fazem parte do trabalho de qualquer programador. A mensagem de erro do Python, chamada de *traceback*, pode ser longa, mas **a informação mais importante costuma estar na última linha**.



- ① **Onde** aconteceu: a seta `---->` aponta a linha do código com problema.
- ② O **tipo** do erro: `NameError`, um problema com um nome.
- ③ A **explicação**: `name 'total' is not defined`, ou seja, o nome `total` não está definido.

Os erros mais frequentes no início:

Erro	O que significa	Causa mais comum
`NameError`	O Python não conhece esse nome	Variável não criada, célula não executada ou nome digitado errado
`SyntaxError`	O código não está escrito em "Python válido"	Faltou fechar aspas ou parênteses
`IndentationError`	Os espaços no começo da linha estão errados	Um bloco de código ficou desalinhado
`TypeError`	Operação com tipos de dados incompatíveis	Somar texto com número, por exemplo
`ModuleNotFoundError`	Biblioteca não encontrada	O kernel não é o do `.venv` ou a biblioteca não foi instalada

Sua vez: corrigindo erros

Cada célula abaixo tem um erro **de propósito**. Execute, leia a **última linha** da mensagem, corrija o código na própria célula e execute de novo.

Erro 1 de 3.

```
In [19]: print("Faltou alguma coisa aqui)
```

```
  Cell In[19], line 1
    print("Faltou alguma coisa aqui)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Ver a resposta

A última linha diz `SyntaxError: unterminated string literal`, ou seja, "texto não terminado": as aspas foram abertas antes de `Faltou` e nunca foram fechadas. A correção é `print("Faltou alguma coisa aqui")`.

Erro 2 de 3.

```
In [20]: mensagem = "Web scraping"
         print(mensagen)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[20], line 2
      1 mensagem = "Web scraping"
----> 2 print(mensagen)

NameError: name 'mensagen' is not defined
```

Ver a resposta

A última linha diz `NameError: name 'mensagen' is not defined`: o nome foi digitado errado. A variável criada na linha de cima se chama **`mensagem`**, com **m** no final.

Erro 3 de 3.

```
In [21]: print("Total de livros: " + 1000)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 print("Total de livros: " + 1000)

TypeError: can only concatenate str (not "int") to str
```

Ver a resposta

A última linha diz `TypeError: can only concatenate str (not "int") to str`: não é possível "somar" um texto com um número. Use uma f-string: `print(f"Total de livros: {1000}")`.

2.7 Escrevendo em Markdown

As células de texto usam **Markdown**, uma forma simples de formatar textos com símbolos. Você pode usá-las para fazer as suas próprias anotações nos notebooks do curso.

Você escreve	E aparece
`# Título`	um título grande
`## Subtítulo`	um título menor
`**negrito**`	**negrito**
`*itálico*`	*itálico*
`- item`	• item de lista
`1. item`	1. item numerado
`` `código` ``	`código`
`[Python](https://www.python.org)`	Python

▶️ **Experimente:** dê um **duplo clique** nesta célula para ver como ela foi escrita por dentro. Depois aperte **`Shift + Enter`** para ela voltar a ficar formatada.

Sua vez: a sua primeira célula de texto

1. Clique nesta célula e aperte `Esc`; depois aperte `B`, para criar uma célula abaixo, e `M`, para transformá-la em Markdown (os atalhos são explicados na seção 2.8).
2. Aperte `Enter` para começar a digitar e escreva: um título com o seu nome, uma lista com 3 sites dos quais você gostaria de coletar dados e uma palavra em **negrito**.
3. Aperte `Shift + Enter` para ver o resultado formatado.

Ver uma resposta possível

```
# Ana Souza

- Site de uma livraria
- Previsão do tempo
- Classificados de imóveis

Objetivo: montar um **dataset** de preços.
```

2.8 Atalhos de teclado: modo de comando e modo de edição

O notebook tem dois "modos":

- **Modo de edição**: você está **digitando dentro** de uma célula (aparece o cursor piscando). Entre nele apertando `Enter` ou clicando dentro da célula.
- **Modo de comando**: a célula está **selecionada**, mas você não está digitando nela. Entre nele apertando `Esc`. Nesse modo, algumas teclas funcionam como atalhos:

Tecla (no modo de comando)	Ação
`A`	Cria uma célula **acima** (*above*)
`B`	Cria uma célula **abaixo** (*below*)
`D` `D` (duas vezes)	**Apaga** a célula
`M`	Transforma em célula de texto (**Markdown**)
`Y`	Transforma em célula de **código**
`↑` `↓`	Navega entre as células

E, em qualquer modo, `Ctrl + S` salva o notebook.

▶️ **Experimente:** clique nesta célula, aperte `Esc` e depois `B`. Uma célula nova aparece abaixo. Apague-a com `D` `D`.

2.9 Instalando bibliotecas de dentro do notebook

Às vezes queremos instalar uma biblioteca sem sair do notebook. Para isso existe o `%pip`. Ele funciona igual ao `pip` do terminal, com uma vantagem: instala **no mesmo Python que está executando o notebook** (o kernel), evitando que a biblioteca seja instalada em outro Python.

▶️ Execute a célula abaixo. Ela não instala nada: o `show` só **mostra informações** sobre uma biblioteca que já está instalada.

In [22]:
```
%pip show requests
```

```
Name: requests
Version: 2.34.2
Summary: Python HTTP for Humans.
Home-page:
Author:
Author-email: Kenneth Reitz <me@kennethreitz.org>
License: Apache-2.0
Location: C:\Users\aluno\Documents\curso-web-scraping\.venv\Lib\site-packages
Requires: certifi, charset_normalizer, idna, urllib3
Required-by: jupyterlab_server, requests-file, tldextract
Note: you may need to restart the kernel to use updated packages.
```

Aparecem o nome, a versão e uma descrição do `requests`. Repare na linha `Location`: ela deve apontar para dentro da pasta `.venv` do curso.

> 💡 Em tutoriais na internet, você pode encontrar `!pip install ...` (com exclamação). O `!` executa qualquer comando de terminal, mas pode acabar usando um Python diferente do kernel. Prefira sempre o `%pip`.

Sua vez: informações de uma biblioteca

Mostre as informações do `parsel`. Na linha `Requires`, aparecem as bibliotecas de que ele depende: o `lxml` está entre elas?

In [23]:
```
# Escreva o seu código aqui
```

Ver a resposta

```
%pip show parsel
```

2.10 Teste final do ambiente

As quatro células a seguir conferem, uma de cada vez, se o ambiente está pronto para os próximos módulos. Cada uma mostra ✅ quando está tudo certo, ou ❌ e a seção a revisar. Não é preciso entender todo o código agora: ele será explicado ao longo do curso.

▶️ **1. A versão do Python.** O módulo `sys`, que vem com o Python, informa a versão em uso:

In [24]:
```
import sys

versao = sys.version.split()[0]
if sys.version_info >= (3, 11):
    print("✅ Python", versao)
else:
    print("❌ Python", versao, "é antigo demais. Instale o Python 3.14 (seção 1.4).")
```

```
✅ Python 3.14.8
```

▶️ **2. O ambiente virtual.** Dentro de um ambiente virtual, `sys.prefix` (a pasta do Python em uso) é diferente de `sys.base_prefix` (a pasta do Python instalado no computador):

In [25]:
```
from pathlib import Path

if sys.prefix != sys.base_prefix:
    print("✅ Ambiente virtual em uso:", Path(sys.prefix).name)
else:
    print("❌ O notebook não está usando o .venv. Escolha o kernel certo (seção 1.8).")
```

```
✅ Ambiente virtual em uso: .venv
```

▶️ **3. As bibliotecas do curso.** A função `version()`, do módulo `importlib.metadata`, devolve a versão de uma biblioteca instalada, e dá erro se ela não estiver instalada. A célula confere as bibliotecas usadas nos Módulos 01 a 05:

In [26]:
```python
from importlib.metadata import PackageNotFoundError, version

for biblioteca in ["notebook", "requests", "parsel", "lxml"]:
    try:
        print("✅", biblioteca, version(biblioteca))
    except PackageNotFoundError:
        print("❌", biblioteca, "não está instalada. Confira a seção 1.7.")
```

```
✅ notebook 7.6.3
✅ requests 2.34.2
✅ parsel 1.12.1
✅ lxml 6.1.3
```

▶️ **4. A internet, com uma pequena raspagem.** A célula baixa a página inicial de um site de treino e extrai a primeira frase dela. O código será explicado nos Módulos 03 e 04.

In [27]:
```python
import requests
from parsel import Selector

resposta = requests.get("https://quotes.toscrape.com/", timeout=20)
pagina = Selector(text=resposta.text)
print("✅ O site respondeu com o código", resposta.status_code)
print(pagina.css("span.text::text").get())
```

```
✅ O site respondeu com o código 200
“The world as we have created it is a process of our thinking. It cannot be changed without changing our thi
nking.”
```

Se as quatro células mostraram ✅ e a última mostrou uma frase, o ambiente está pronto para o curso. Se a última célula mostrar um erro com `ConnectionError`, confira a sua conexão com a internet.

Repare na última célula: em poucas linhas, o código **baixou uma página da internet e extraiu uma frase dela**. Esse é o princípio do web scraping, que será desenvolvido nos próximos módulos.

Problemas comuns e como resolver

Sintoma	Causa provável	Solução
No terminal: `Python was not found` ou `O termo 'python' não é reconhecido...`	Python não instalado, ou instalado fora do PATH	Instale pela seção 1.4. No Windows, prefira o comando `py`. Feche e abra o terminal depois de instalar.
Digitar `python` abre a **Microsoft Store**	Um atalho do Windows aponta para a loja	Use `py` no lugar de `python`. Ou, no menu Iniciar, pesquise **aliases**, abra *Gerenciar aliases de execução de aplicativo* e desligue os atalhos do "Instalador de aplicativo" para `python.exe` e `python3.exe`.
`...a execução de scripts foi desabilitada neste sistema`	Proteção do PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (seção 1.6)
`pip` ou `jupyter` "não é reconhecido"	O ambiente virtual não está ativado	Ative o `.venv` (seção 1.6). Lembre-se: é preciso ativar em todo terminal novo.
Erros ao instalar, como `Failed building wheel` ou `Microsoft Visual C++ ... is required`	Versão do Python nova demais (3.15, por exemplo)	Use o Python 3.14 (seções 1.3 e 1.4) e recrie o `.venv`: apague a pasta e repita as seções 1.6 e 1.7.
No notebook: `ModuleNotFoundError: No module named 'requests'`	O kernel não é o Python do `.venv`	Jupyter: feche e abra o Jupyter **com o `(.venv)` ativo**. VS Code: **Select Kernel** e escolha o `.venv`.
O navegador não abriu sozinho	Acontece em alguns computadores	Copie o endereço `http://localhost:8888/tree?token=...` que aparece no terminal e cole no navegador.
A última célula do teste final mostra `ConnectionError`	Sem conexão com a internet, ou o site de treino está fora do ar	Confira a conexão e abra `https://quotes.toscrape.com/` no navegador
A célula fica em `[*]` para sempre	O código travou ou está esperando algo	Clique em ■ (*Interrupt*) ou reinicie o kernel.
Aviso de *Kernel died* ou *Dead kernel*	O processo do kernel parou	Reinicie o kernel. Se repetir, feche tudo e abra de novo.
Não tenho permissão para instalar programas	Computador da escola ou do trabalho	Use o **Google Colab** (colab.research.google.com), que executa notebooks no navegador. Envie o `.ipynb` em *File → Upload notebook* e instale o que faltar com `%pip install parsel`. As imagens do material podem não aparecer por lá.

Retomada do Módulo 01

Resumo do que foi visto neste módulo:



O que você aprendeu

Na Parte 1, você montou o ambiente:

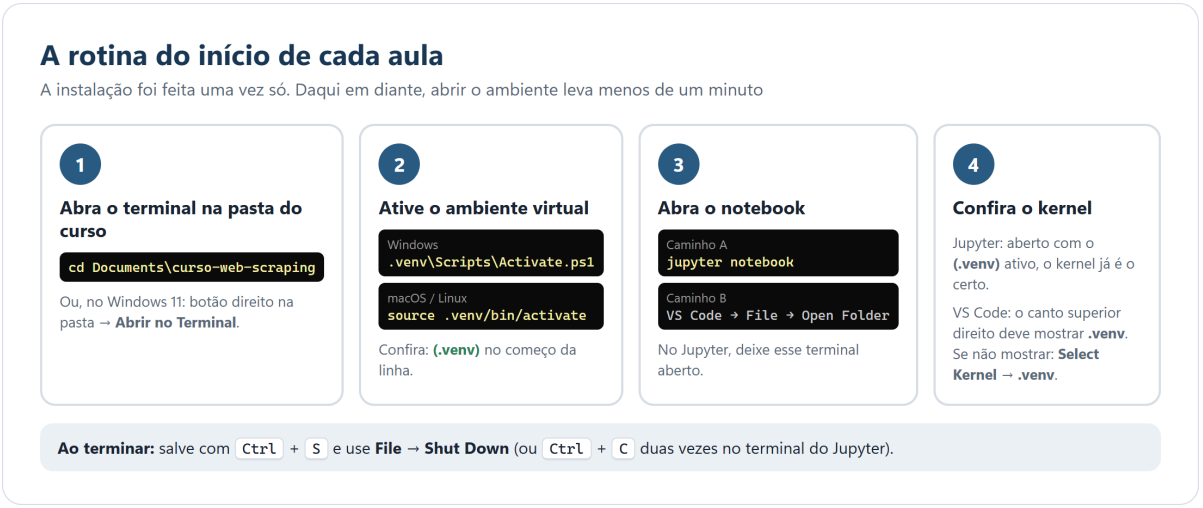
- O **terminal** recebe comandos digitados. O *prompt* mostra a pasta atual; `cd` entra em uma pasta e `dir` ou `ls` lista o que tem nela.
- O **Python** é instalado uma vez no computador. Usamos a versão **3.14** porque todas as bibliotecas do curso já funcionam nela.
- O **ambiente virtual** (`.venv`) isola as bibliotecas do curso: é criado com `-m venv .venv` e precisa ser **ativado em todo terminal novo**.
- O **pip** instala bibliotecas, e o `requirements.txt` é a lista de tudo o que o curso precisa.
- O **Jupyter** (no navegador) e o **VS Code** abrem notebooks. O importante é que o **kernel** seja o Python do `.venv` .

Na Parte 2, você aprendeu a usar o notebook:

- Notebooks têm células de **código** e de **texto (Markdown)**. `Shift + Enter` executa e avança.
- O **kernel** guarda as variáveis na **ordem em que as células são executadas**, e o contador `[n]` mostra essa ordem.
- **Reiniciar** o kernel apaga a memória. *Restart and Run All* executa tudo de novo, em ordem.
- Em uma mensagem de erro, **leia a última linha primeiro**.
- O `%pip` instala bibliotecas de dentro do notebook, no ambiente certo.

A rotina do início de cada aula

Esta sequência se repete no início de cada aula:



Teste seus conhecimentos

Clique em cada pergunta para ver a resposta.

1. Abri um terminal novo, e o comando `jupyter` "não é reconhecido". O que eu esqueci?
Provavelmente de **ativar o ambiente virtual** (`.venv\Scripts\Activate.ps1` no Windows ou `source .venv/bin/activate` no macOS e no Linux). A ativação vale só para a janela em que foi feita.

2. Para que serve o arquivo `requirements.txt` ?
É a lista das bibliotecas do curso. Com `pip install -r requirements.txt` , o pip instala todas de uma vez.

3. Uma célula mostra `[7]` , e a célula logo abaixo dela mostra `[3]` . O que isso quer dizer?
Que a célula de baixo foi executada **antes** (foi a 3ª execução) e a de cima, **depois** (a 7ª). O kernel segue a ordem de execução, não a ordem em que as células aparecem na tela.

4. Recebi `ModuleNotFoundError: No module named 'parsel'` no notebook, mas instalei tudo. Por quê?
O notebook está usando um **kernel diferente** do `.venv` , onde as bibliotecas foram instaladas. Escolha o kernel do `.venv` (seção 1.8).

5. Qual a diferença entre `Shift + Enter` e `Ctrl + Enter` ?
Os dois executam a célula. O `Shift + Enter` **avança** para a próxima célula; o `Ctrl + Enter` **continua** na mesma.

Glossário

Termo	Significado
Terminal	Programa para operar o computador por comandos digitados
Prompt	O começo da linha do terminal, que mostra a pasta atual
Python	A linguagem de programação e o programa que executa o código (o interpretador)
Biblioteca (ou pacote)	Código pronto que adiciona novas funções ao Python
pip	O instalador de bibliotecas do Python
Ambiente virtual (venv)	Pasta que isola as bibliotecas de um projeto
Notebook (`.ipynb`)	Documento com células de código e de texto
Kernel	O Python que executa as células e guarda as variáveis na memória
Markdown	Forma simples de formatar textos usando símbolos
Traceback	A mensagem de erro do Python

Próximo módulo

O **Módulo 02: Fundamentos da Web** explica o que acontece quando um endereço é digitado no navegador: o que é o **HTTP**, como o seu computador conversa com um **servidor**, por que uma URL se parece com o caminho de uma **pasta** e como o **HTML** organiza o conteúdo de uma página. Esses conceitos são a base de todo o restante do curso.

Referências

Materiais consultados na elaboração deste módulo, agrupados pela parte em que foram usados.

Parte 1: instalação

- Documentação do Python: Usando Python no Windows, inclusive o Python install manager; venv, sobre ambientes virtuais.
- User Guide do pip, sobre `pip install` e arquivos de requisitos (em inglês).
- Documentação do Jupyter Notebook e Notebooks Jupyter no VS Code (em inglês).

Parte 2: o notebook e o Python

- Tutorial oficial do Python, em português: a leitura recomendada para aprofundar variáveis, contas e erros.
- Documentação do Python: importlib.metadata, usado no teste final.